

Algorithms Visualizer

A practical guide to searching, sorting, learning, and quiz practice

WATCH THE PROCESS

Understand the logic.

Visualize operations, compare algorithm costs, and practice core concepts with saved quizzes.

SEARCH / SORT / LEARN

Project overview

The Algorithms Visualizer is an educational web application for exploring common search and sorting algorithms. It animates operations on numeric arrays, explains algorithm decisions, compares time and space complexity, and includes interactive quiz practice.

Main features

Visualizer: Load arrays and step through comparisons, swaps, search decisions, and sorting operations.

Learn: Explore each algorithm's meaning, why and when to use it, how it works, formulas, and examples.

Compare: Review best, average, and worst-case time and extra-space costs side by side.

Quiz: Practice with timed sets, saved progress, feedback, score summaries, and answer explanations.

Learning objectives

- Trace: connect each change in the visualization to the rule that caused it.
- Analyze: explain how time and memory needs grow as input size n increases.
- Choose: select a method based on input order, size, stability, and memory constraints.

Quiz progress and history are stored in `localStorage` in the current browser profile. This is local browser storage, not a cloud account or cross-device sync.

Searching algorithms

Find a target value in a collection

Linear Search

Meaning: Checks values sequentially until it finds the target or reaches the end.

Why use it: It works with unsorted input and is straightforward for small collections or sequential data structures.

How it works: Start at index 0; compare each value with the target; return the index at a match, otherwise report not found.

Formula and cost: Best $O(1)$; average and worst $O(n)$. Space $O(1)$. For an equally likely successful position, expected comparisons are $(n+1)/2$.

Example: Scan a list of names from the first entry onward.

Binary Search

Meaning: Repeatedly halves the remaining search range.

Requirement: The values must be sorted; sorting first may have a cost of its own.

How it works: Compare target with the middle value. Keep only the half that could contain the target and repeat.

Formula and cost: $T(n) = T(n/2) + O(1)$, yielding $O(\log n)$ time. Best $O(1)$; iterative extra space $O(1)$.

Example: Find a word in a dictionary by narrowing the range around its middle.

Choosing between them

Linear Search can inspect an unsorted list but may examine every item. Binary Search needs sorted, indexable data; in exchange, each comparison discards about half of the remaining candidates.

- Use Linear Search: when the collection is small, unsorted, or only traversable sequentially.
- Use Binary Search: when data is sorted and many lookups can benefit from logarithmic search.

Simple sorting algorithms

Comparison-based methods that are easy to trace

Bubble Sort

Meaning: Repeated adjacent comparisons move out-of-order values toward their final positions.

How it works: Sweep through the unsorted range, swapping neighbors if $\text{left} > \text{right}$; repeat until sorted.

Why and when: Easy to learn, but generally unsuitable for large inputs.

Formula and cost: Up to $n(n-1)/2$ comparisons across full passes. Best $O(n)$ with early-exit optimization; average/worst $O(n^2)$; space $O(1)$.

Example: Neighbors swap until people in a line are ordered by height.

Selection Sort

Meaning: Grows a sorted prefix by selecting the smallest value from the remaining suffix.

How it works: At position i , find the minimum from i to $n-1$ and swap it into position i .

Why and when: Can limit writes, but still performs many comparisons; useful for small inputs and teaching.

Formula and cost: Comparisons = $n(n-1)/2$ in every case; time $O(n^2)$; extra space $O(1)$.

Example: Repeatedly select the smallest remaining item and place it next.

Insertion Sort

Meaning: Maintains a sorted prefix and inserts each next value in its correct place.

How it works: Hold the next value, shift larger prefix values right, then insert the held value.

Why and when: Works especially well on small or nearly sorted inputs; often used in hybrid sorts.

Cost: Best $O(n)$, average/worst $O(n^2)$, extra space $O(1)$. Reverse order causes many shifts.

Example: Insert each playing card into its proper position in a sorted hand.

Advanced sorting algorithms

Divide-and-conquer and heap-based approaches

Merge Sort

Meaning: A stable divide-and-conquer sort that recursively sorts halves and merges them.

How it works: Split until one-item arrays remain; merge ordered parts by repeatedly taking the smaller front item.

Recurrence: $T(n) = 2T(n/2) + O(n)$, so best, average, and worst time are $O(n \log n)$.

Why and when: Predictable performance, stability, and external sorting; requires $O(n)$ extra memory.

Example: Sort smaller groups independently, then merge the ordered groups.

Quick Sort

Meaning: Partitions values around a pivot, then sorts each side recursively.

How it works: Choose a pivot; partition smaller and larger values; apply the process to each partition.

Cost: Balanced partitions: $O(n \log n)$. Repeatedly unbalanced partitions: $O(n^2)$. Typical stack space $O(\log n)$, worst $O(n)$.

Why and when: Often fast in memory with modest overhead. Stability and worst-case behavior depend on implementation.

Example: Use a reference value to divide a set into lower and higher groups.

Heap Sort

Meaning: Uses a max heap to repeatedly select the largest remaining value.

How it works: Build a max heap; swap the root with the last unsorted item; shrink the heap and restore heap order.

Cost: Heap construction $O(n)$; repeated extractions $O(n \log n)$; total $O(n \log n)$ in every case; extra array space $O(1)$.

Why and when: Useful when worst-case time guarantees and low extra memory are important; not stable.

Example: A priority structure exposes its largest value at the root.

Complexity reference

Big-O describes growth as input size n increases; constants are omitted.

Algorithm	Best	Average	Worst	Extra space
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$ iterative
Bubble Sort	$O(n)^*$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ typical
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

* Bubble Sort has $O(n)$ best-case time when it stops early after a no-swap pass. Quick Sort uses $O(\log n)$ stack space with balanced partitions, but may use $O(n)$ with highly unbalanced ones.

Reading the notation

- $O(1)$: constant work as input grows.
- $O(\log n)$: often indicates repeatedly halving the remaining problem.
- $O(n)$: work grows in proportion to the number of items.
- $O(n \log n)$: common growth rate for efficient comparison sorting.
- $O(n^2)$: often comes from pairwise or nested comparisons; expensive for large n .

Important terms

- Stable sort: preserves relative order of records with equal keys. Standard Merge and Bubble Sort are stable; Quick and Heap Sort are not.
- Auxiliary space: extra working memory beyond the input.
- Best / average / worst case: cost for favorable, typical, and most costly input arrangements.

Learn and practice

A study framework and quiz feature guide

Learn section

Each algorithm topic addresses the questions learners need to connect concepts with execution:

Learning framework

Meaning: A plain-language definition of the algorithm.

Why use it?: Its strengths, trade-offs, and conditions where it is useful.

How it works: The main steps that lead to the result.

Formula and complexity: A recurrence or comparison-count idea, plus time and extra-space costs.

Example and use: A familiar analogy and practical input scenarios.

Interactive quiz

Quiz features

20 sets: Each set has 10 multiple-choice algorithm questions.

10-minute timer: Each attempt is timed. When time expires, the attempt is automatically completed.

Feedback: After a choice, the app highlights the answer and explains why it is correct.

Navigation: Move between questions and change selected answers before finishing; unanswered items count as skipped.

Scoring: Correct answers earn one point; wrong and skipped items do not. $\text{Accuracy} = (\text{correct} / \text{total questions}) \times 100$.

Results and review: Shows score, accuracy, correct/wrong/skipped counts, time remaining, each response, the correct answer, and explanation.

Persistence: Unfinished progress resumes after refresh. Completed history is grouped by set and can be cleared.

Storage and privacy

Quiz data is saved in localStorage on the current browser profile and device. It is not sent to a server or synchronized across browsers. Clearing site data or switching devices may make saved progress unavailable.